

Document Information

This document and the Software Products that it describes are protected by copyright law and international treaties. Unauthorized use, reproduction or distribution of this document, or any portion of it, may result in severe civil and criminal penalties, and will be prosecuted to the maximum extent possible under the law.

The Software Products described in this document are licensed strictly in accordance with a separate Software System License Agreement, granted by Rithmic, LLC, which contains restrictions on use, reverse engineering, disclosure, confidentiality and other matters.

Information in this document, as well as the features and specifications of the Software Products described by this document, are subject to change without notice. Rithmic, LLC, makes no claims as to the accuracy or completeness of any information contained herein. Rithmic, LLC, is not responsible for any typographical errors contained in this document.

Copyright (c) 2024 Rithmic, LLC.

Rithmic™, R | Trade Execution Platform™, R | Trader™, R | Trader Pro™, R | Manager™, R | API™, R | API+™, R | DiamondAPI™, and R | API+.NET™ are trademarks of Rithmic, LLC, in the United States, other countries, or both.

Table Of Contents

- **Preface**
- **What is R | Trade Execution Platform?**
- **What is R | API+?**
- **Operating Systems, Frameworks, Compilers and Target Platforms**
- **Installation and File List**
- **Building A C++ Rithmic API Application**
 - **Building C++ On Windows Using Visual Studio**
 - **Building C++ On Linux Using GCC/g++**
 - **Building C++ On Darwin Using GCC/g++**
- **Building A .NET Rithmic API Application**
- **Organization Of Rithmic API Classes**
 - **There are three main categories of classes in the Rithmic APIs :**
 - **The dump() Method**
- **Configuring A Rithmic API Application**
 - **Configuration Files**
 - **Conformance and Fees**
- **Max Session Counts**
- **Using R | Trader Pro as a Plug-In Host**
- **Connecting to Multiple Rithmic Systems Simultaneously Using Environments**
 - **Background**
 - **Multiple Environments**
 - **Programming Interface**
 - **Prior Behavior With A Single Environment**
 - **Comments**
- **Connections**

- **Connect Points**
 - **Aggregated Data Market Data**
 - **Gateways and Points-Of-Presence**
 - **Best Practices**
 - **Connection Management**
 - **Establishing And Maintaining State With The Snapshot-Update Pattern**
 - **Strings And tsNCharcb (C++ Only)**
 - **Why use the tsNCharcb for representing strings?**
 - **How does one convert a C++ string into a tsNCharcb?**
 - **How does one convert a tsNCharcb into a C++ string?**
 - **Do I need to initialize both the pData and the iDataLen of each tsNCharcb?**
 - **Who owns the memory pointed to by tsNCharcb::pData?**
 - **On Timestamps (ssboe, usecs and nsecs)**
 - **On Threads**
 - **Threads In C++ API**
 - **Threads In .NET API**
 - **Error Handling Conventions**
 - **Error Codes**
 - **Exception Classes**
 - **Why And Where Errors Might Occur**
 - **The 'aiCode' Out Parameter (C++ only)**
 - **Examples Of Errors**
 - **Error Handling In User Implemented Callbacks**
 - **Basic Steps of a Program that Incorporates R | API**
 - **Feedback And Bug Reports**
 - **Contact Info**
-

Preface

This Programmer's Guide is a high level description of R | API along with technical background information. It is intended for users new to Rithmic's APIs looking for a basic overview. For additional information please consult other resources, such as documentation for each class/method, sample code, and/or the FAQ.

Suggestions for improvement of this document are always welcome.

What is R | Trade Execution Platform?

The R | Trade Execution Platform is software that implements a trading platform with market data, risk management and order handling functionality. There are many instances of the R | Trade Execution Platform. Each instance is referred to as a "system", or "environment". Some are private systems, and others exist under different brands, or are available through a particular FCM or broker.

Each of these systems appear as values in the "System" drop down box of the R | Trader login screen. Programmatically, one specifies different parameters to the REngine constructor to access different Rithmic systems.

Each of these systems serves a different purpose. For example,

Rithmic Test : The test environment is called Rithmic Test. Exchanges typically provide a test environment in which test orders are placed against a test matching engine, and resultant market data is published. This

environment uses these test environments. This environment can appear strange because the market data can be unrealistic, and is often driven by what (test) orders are resting on the matching engine.

Rithmic O1 : There is a production environment called Rithmic O1 which uses live market data and live matching engines.

Rithmic Paper Trading : This environment uses live market data, but with a simulator for a matching engine.

Each installation has its own administrative organizations responsible for creating or maintaining that installation's user ids and trading accounts. Typically, whoever issues a user id also handles the administration of that user id and any associated trading accounts. This administrative role includes configuring market data entitlements for user ids and risk management settings for trading accounts. A user id must be entitled to market data from a particular exchange to receive the market data stream. There may also be differences in whether a user can only see the top of the book, or the entire market depth. If a trading account does not have risk management settings configured for a particular instrument or set of instruments, users will not be able to place orders for those instruments on that account. Each user id has the ability to trade on zero to many trading accounts.

What is R | API+?

R | API+ (sometimes called R | API or RAPI) is collection of libraries and configuration files constituting a programmatic interface for accessing services provided by Rithmic's R | Trade Execution Platform. By incorporating R | API into your software, you (and your software) may retrieve real-time and historical market data and reference data for various financial securities and contracts, and you may place and manage orders for those securities and contracts. R | API also allows you to obtain account and margin balances, order history and p&l and position information. The specific accounts you may view and access, the instruments you may trade, the market data you may view and the number of concurrent connections you may maintain to R | Trade Execution Platform is subject to restrictions and permissions established by your Futures Commission Merchant (FCM), your Broker-Dealer or your Introducing Broker (IB), as the case may be.

C++ vs. .NET

There are two compilations of R | API, one for C++ and the other for .NET. The interface for the C++ compilation does not make use of 'modern' C++ and should be accessible to anyone with knowledge of C or 'classic' C++. Furthermore, the C++ classes do not necessarily adhere to standard STL conventions and should be used with care in an STL environment.

As much as possible the feature set and interface exposed by both compilations are the same, but because they are built on different underlying libraries, some differences remain, including obvious differences such as the use of properties in .NET as well as more subtle differences such as the threading model used for handling events. In general, the behavior of the two compilations is kept as close as possible.

The .NET compilation of R | API is a pure .NET library. It does not contain unmanaged code.

R | API+ and R | Diamond Cutter

There are two variations of R | API, both using the same basic object model. R | API+ provides trading infrastructure services. R | Diamond Cutter includes everything R | API+ has, with additional functionality for ultra low latency applications. R | Diamond Cutter is only offered in C++.

R | API (no plus) has been discontinued.

Operating Systems, Frameworks, Compilers and Target Platforms

The Rithmic APIs make use of a set of libraries licensed from Omnesys Technologies, Inc. These libraries and the Rithmic APIs have been built on several operating systems and frameworks with specific compilers for use on specific target platforms as indicated below. Please be sure to build your software using only the compilers and frameworks listed below on machines running the associated operating systems. Support will not be provided for software built any other way and support will not be provided for software run on any other operating systems and/or frameworks. Also, support will not be provided for software run on any operating system and/or framework emulators.

The C++ compilation of Rithmic APIs have been built as follows:

Build Operating System	Compiler	Target Platform
Darwin 10.8.0 for darwin-10 (64-bit darwin)	gcc/g++ 4.1.2	Darwin 10.8.0 for darwin-10 (64-bit darwin)
Darwin 20.6 for darwin-20.6-arm64 (M1)	gcc/g++ (clang-1300.0.29.30)	Darwin 20.6 for darwin-20.6-arm64 (M1)
RHEL 6.3 for linux-gnu-2.6.32-x86_64 (64-bit linux)	gcc/g++ 4.4.6	RHEL 6.3 for linux-gnu-2.6.32-x86_64 (64-bit linux)
RHEL 7.0 for linux-gnu-3.10.0-x86_64 (64-bit linux)	gcc/g++ 4.8.2	RHEL 7.0 for linux-gnu-3.10.0-x86_64 (64-bit linux)
RHEL 8.5 for linux-gnu-4.18-x86_64 (64-bit linux)	gcc/g++ 8.5.0	RHEL 8.5 for linux-gnu-4.18-x86_64 (64-bit linux)
Windows 10 Professional (64-bit)	Visual Studio 2015 Professional	Windows 10 Professional (32-bit)

The .NET compilation of Rithmic APIs have been built with a target framework of .NET 3.5 as follows:

Build Operating System	Compiler	Target Platform
Windows (64-bit)	Visual Studio 2010 Professional	AnyCPU on .NET Framework 3.5

Installation and File List

Rithmic APIs are distributed as a compressed set of files (a zip archive) which can be extracted using standard utilities. The top level directory within the zip archive is the version number of the API. Beneath this directory are documentation and assorted files necessary for building programs that incorporate the API.

Files in the C++ Compilation

File or Sub-directory	Description
darwin-10/lib/	Directory containing libraries to link against when building for darwin-10
darwin-20.6/lib/	Directory containing libraries to link against when building for Apple's M1 architecture
documentation.html	Start page for html documentation (Redirects to doc/html/index.html)
etc/	Directory containing SSL/TLS files

include/	Directory containing the include file
linux-gnu-2.6.32-x86_64/lib/	Directory containing libraries to link against when building for linux-gnu-2.6.32-x86_64
linux-gnu-3.10.0-x86_64/lib/	Directory containing libraries to link against when building for linux-gnu-3.10.0-x86_64
linux-gnu-4.18-x86_64/lib/	Directory containing libraries to link against when building for linux-gnu-4.18-x86_64
samples/	Directory containing sample code
win10/lib/	Directory containing libraries to link against when building for Windows 10
Release.Notes	Change log

Files in the .NET Compilation

File or Sub-directory	Description
documentation.html	Start page for html documentation (Redirects to doc/html/index.html)
samples/	Directory containing sample code.
win10/lib/	Directory containing libraries to link against when building for Windows (.NET Framework 3.5).
Release.Notes	Change log

Building A C++ Rithmic API Application

In the samples directory there are VS2010 and VS2015 sample solutions. The VS2015 sample solution should be opened, then converted, if using a later version of Visual Studio. Also, in the comments at the top of each source file, there is a compile/link line one can use in unix-like operating systems. There is also information about configuring, building and running the sample code in the [Quick Start](#).

To compile an application which uses a Rithmic API library, your compiler must know the location of the header file. The header file name will be "RApiPlus.h" or "RDiamondApi.h", depending on which product you are using.

Building C++ On Windows Using Visual Studio

Compiling On Windows

Code should be generated with the run-time libraries set to Multi-threaded (/MT), Multi-threaded Debug (/MTd), Multi-threaded DLL (/MD), or Multi-threaded Debug DLL (/MDd). Code should be compiled with run-time type information enabled (/GR).

If you are compiling the sample code, you should define "WinOS".

Linking On Windows

To link an application under Windows, you will need to link with a number of libraries included in the [RApi](#) package under the win7\lib directory for Visual Studio 2010 libraries, and the win10\lib directory for Visual Studio 2015 libraries. The libraries built with Visual Studio 2015 should work with later versions of Visual Studio. All the libraries are named with different suffixes indicating the code generation flag used when compiling :

- *_mdd.lib indicates /MDd on 32-bit platforms

- *_md.lib indicates /MD on 32-bit platforms
- *_mtd.lib indicates /MTd on 32-bit platforms
- *_mt.lib indicates /MT on 32-bit platforms
- *_mdd64.lib indicates /MDd on 64-bit platforms
- *_md64.lib indicates /MD on 64-bit platforms
- *_mtd64.lib indicates /MTd on 64-bit platforms
- *_mt64.lib indicates /MT on 64-bit platforms

So, if the code was compiled using Multi-threaded DLL (/MD) for the 32-bit platform, the following libraries would be used to link :

- kit_md.lib
- apistb_md.lib
- api_md.lib
- OmneEngine_md.lib
- OmneChannel_md.lib
- OmneStreamEngine_md.lib
- RApiPlus_md.lib (or RDiamondApi_md.lib)

You will also need these SSL/TLS libraries from OpenSSL (also included in the **RApi** package) :

- libcrypto_md.lib
- libssl_md.lib

And these zlib libraries (also included in the **RApi** package) :

- zlib_md.lib

You may also need to specify these system libraries :

- ws2_32.lib
- iphlpapi.lib
- psapi.lib
- crypt32.lib

If you are using the /NODEFAULTLIB switch, you will need the following libraries:

- kernel32.lib
- gdi32.lib
- advapi32.lib
- user32.lib
- bcrypt.lib

You also need to link with The C (and possibly C++) runtime libraries. The C runtime library names have changed between Visual Studio 2010 and Visual Studio 2015.

For Visual Studio 2010 C runtime libraries, use :

- msvcr.lib (/MD) or libcmtd.lib (/MT) or msvcrtd.lib (/MDd) or libcmtd.lib (/MTd)
- OLDNAMES.lib

For Visual Studio 2015 C runtime libraries use :

- ucrt.lib (/MD) or libucrt.lib (/MT) or ucrttd.lib (/MDd) or libucrttd.lib (/MTd)
- vcruntime.lib (/MD) or libvcruntime.lib (/MT) or vcruntime.lib (/MDd) or libvcruntime.lib (/MTd)
- msvcr.lib (/MD) or libcmtd.lib (/MT) or msvcrtd.lib (/MDd) or libcmtd.lib (/MTd)

- legacy_stdio_definitions.lib
- OLDNAMES.lib

If you are using the standard C++ runtime library, you may need the following libraries:

- msvcprt.lib (/MD) or
 - libcpmt.lib (/MT)
 - msvcprtd.lib (/MDd) or
 - libcpmtd.lib (/MTd)
-

Building C++ On Linux Using GCC/g++

Each of the sample code source files includes in the comments at the top of the file, a build line for that sample. You shouldn't need to modify it if your working directory is the ./samples directory and you are using the RApiPlus.cpp package.

If you are using the RDiamondApi.cpp package, you will have to edit the name of the **RApi** library you link against.

Compiling On Linux

Internal testing is done using the following compiler flags :

- -DLINUX
- -D_REENTRANT
- -Wall
- -Wno-sign-compare
- -Wno-write-strings
- -Wpointer-arith
- -Winline
- -Wno-deprecated
- -fno-strict-aliasing

Linking On Linux

To link an application under linux, you will need to link with the following libraries (included in the **RApi** package) :

- RApiPlus-optimize (or RDiamondApi-optimize)
- OmneStreamEngine-optimize
- OmneChannel-optimize
- OmneEngine-optimize
- _api-optimize
- _apipoll_stubs-optimize
- _kit-optimize

For RHEL 6, you may also need to link with the following system, SSL and z libraries.

Some are included in the zip file in case one wishes to use them. :

- ssl (included)
- crypto (included)
- krb5
- k5crypto
- com_err

- resolv
- m
- pthread
- rt
- z

This compile/link line for SampleMD.cpp using R | API+ works on 64-bit linux of the 2.6.32 kernel (RHEL 6.x). You shouldn't need to modify anything if your working directory is the ./samples directory.

```
g++ -O3 -DLINUX -D_REENTRANT -Wall -Wno-sign-compare -Wno-write-strings -Wpointer-arith -Winline
```

For RHEL 7, the list is similar, but shorter. If the linker claims it "cannot find -ld", you may need to install the package "libzip-devel".

- ssl (included)
- crypto (included)
- pthread
- rt
- z

This compile/link line for SampleMD.cpp using R | API+ works on 64-bit linux of the 3.10.0 kernel (RHEL 7.x). You shouldn't need to modify anything if your working directory is the ./samples directory.

```
g++ -O3 -DLINUX -D_REENTRANT -Wall -Wno-sign-compare -Wno-write-strings -Wpointer-arith -Winline
```

Building C++ On Darwin Using GCC/g++

Compiling On Darwin

Internal testing is done using the following compiler flags :

- -D_REENTRANT
- -Wall
- -Wno-sign-compare
- -fno-strict-aliasing
- -Wpointer-arith
- -Winline
- -Wno-deprecated
- -Wno-write-strings

Linking On Darwin

To link an application under Darwin, you will need to link with the following libraries (included in the **RApi** package) :

- RApiPlus-optimize (or RDiamondApi-optimize)
- OmneStreamEngine-optimize
- OmneChannel-optimize
- OmneEngine-optimize
- _api-optimize
- _apipoll_stubs-optimize
- _kit-optimize

You may also need to link with the following system, SSL and z libraries.
Some are included in the **RApi** package in case one wishes to use them. :

- ssl (included)
- crypto (included)
- krb5
- resolv
- m
- pthread
- z

This compile/link line for SampleMD.cpp works on (64-bit) darwin-10. You shouldn't need to modify anything if your working directory is the ./samples directory.

```
g++ -O3 -D_REENTRANT -Wall -Wno-sign-compare -fno-strict-aliasing -Wpointer-arith -Winline -Wno-dep
```

This compile/link line for SampleMD.cpp works on darwin-20.6-arm64. You shouldn't need to modify anything if your working directory is the ./samples directory.

```
g++ -O3 -D_REENTRANT -Wall -Wno-sign-compare -fno-strict-aliasing -Wpointer-arith -Winline -Wno-dep
```

Building A .NET Rithmic API Application

Building in .NET using Visual Studio is relatively easy.

- Add a reference to rapi.dll to your project or solution.
- Add a **using** directive 'using com.omnesys.omne.om;' to the appropriate source files.
- Add another **using** directive 'using com.omnesys.rapi;' to the appropriate source files.
- Build the application.

Organization Of Rithmic API Classes

There are three main categories of classes in the Rithmic APIs :

- The first category contains the 'action' class. For R | API it is REngine. All actions are initiated by calls to methods of that class. The methods of REngine represent the available services from the infrastructure.
- The second category is for callback classes. Rithmic APIs typically request network services asynchronously. These requests are processed by the infrastructure and the responses are conveyed via methods of callback classes. There are a small number of callback classes :
 - AdmCallbacks for administrative callbacks, and
 - RCallbacks for REngine callbacks (primary callback class for R | API).
- The third category is for 'Info' classes, which convey information specific to a particular callback method. These class names typically end in 'Info', such as RefDataInfo.

The names of an REngine method, its corresponding callback method, and the class used to convey the callback specific information are usually, but not always, similar.

For example :

- REngine::replayPnl() - Requests a snapshot of pnl information for a given account.

- `RCallbacks::PnlReplay()` - The callback method invoked when the response to the request has been received and processed.
- `PnlReplayInfo` - The actual pnl information being returned.

Subscriptions are a notable exception to the naming conventions. Calls to `REngine::subscribe()` and `REngine::subscribeOrder()` can result in a number of different callbacks being invoked.

The `dump()` Method

The `dump()` method that all the Info classes support is a utility method that prints the object contents to stdout. It is deprecated and may be removed in a future release.

Configuring A Rithmic API Application

A Rithmic API application must connect to at least one Rithmic system. (It is possible to connect to more than one system, but that is a more advanced topic not addressed here.) The system to which an `REngine` instance connects to depends on the configuration passed to the `REngine` constructor by `REngineParams`. Along with the `REngine` constructor, the `REngine::login()` routine indicates where (within the Rithmic system) the Rithmic API application will connect, and which what credentials.

Configuration Files

Settings to various Rithmic systems are distributed by the Rithmic operations team in 'connection_params.txt' files, usually made available in your download directory. These files contain information that is needed by the `REngine` constructor and the `REngine::login()` routine to connect to a particular Rithmic system. The prefix of the file name should identify the Rithmic system, and the version number in the file name is an 'as-of' version. It is possible that the version of your R | API is greater than the configuration file's 'as-of' version. The file name might also identify the gateway represented by the configuration. A gateway is a point-of-presence for a Rithmic system. For example, Rithmic 01 has gateways in North America, South America, Europe and Asia. Sometimes users find that one gateway (often geographically closer) is more reliable than another.

The sample code (e.g. `SampleMD` or `SampleOrder`) is hard-coded to connect to Rithmic Test via our Orangeburg gateway. The Rithmic Test configuration files are also available in each download directory. The file name as of this writing is "Rithmic_Test_Orangeburg_rapi_connection_params_10.5.0.0.txt". One can review the relationship between this configuration file and the sample code to see how the file contents map into code.

Rithmic systems and their associated configuration information can be added, modified, or removed from time to time. Vendors of trading applications built on R | API can find it useful to support the ability to dynamically incorporate these kinds of configuration changes.

Conformance and Fees

An R | API application must go through conformance before the connection_params text file containing settings for systems other than Rithmic Test are made available. Most environments requiring conformance make use of live market data and/or live accounts.

Associated with live data or trading can be fees. Fees can vary greatly depending on how many concurrent market data connections a user is entitled to (see **Max Session Counts**), as well as the number of exchanges for which market data is entitled. For users writing their own apps, it is recommended that when going through conformance, you also consult with your broker or fcm to understand any fees associated with using the R | API with these additional Rithmic systems.

To begin conformance, please contact Rithmic at rapi@rithmic.com.

Max Session Counts

The configuration of each user's maximum number of concurrent connections is called the Max Session Count. A user has a Max Session Count for orders, and another Max Session Count for market data. One can view their max session counts using R | Trader Pro by opening the File menu, and selecting User Profile.

When a user exceeds their max session count, the infrastructure will automatically disconnect that user's oldest login session. As a courtesy, the infrastructure will alert the client application prior to severing the connection. This alert appears as a Forced Logout alert in `RCallbacks::Alert()`. When an RAPI client receives a Forced Logout alert, it will log out entirely from the infrastructure, not just the connection on which the forced logout alert arrived.

The Max Session Counts apply per connection. The market data count applies to the market data connect point and to the history connect point. The orders count applies to the trading system connect point and the pnl connect point. So, for example, if a user has a max session count of 2 for market data, they can connect twice to the market data connect point, and twice to the history connect point. If they connect a third time to the market data connect point, their oldest login session will receive a forced logout alert and then will either log out gracefully (if there is time), or have its connection severed by the infrastructure.

Using R | Trader Pro as a Plug-In Host

It is possible to configure an R | API app to be a plug-in client of R | Trader Pro. When an R | API app is a plug-in client, it will connect to R | Trader Pro as its market data and historical data source. This plug-in arrangement allows for a single market data session of R | Trader Pro to be used for multiple R | API plug-in apps.

Use of plug-in functionality conserves the number of concurrent market data connections a user is entitled to, which can reduce applicable market data fees. (See [Max Session Counts](#).) Many users have a max session count for market data of one, and a higher max session count for orders. This allows them to use one connection (plug-in host + N plug-in clients) for market data, and multiple direct connections for orders.

Below are the steps to run a plug-in host and client :

1. Download, install, and then run R | Trader Pro as a plug-in host. You can get a version from the [Rithmic website](#). Versions 15.x.y.z and below do not support plug-in functionality.

1a. From R | Trader Pro's login screen, enable "Allow Plug-ins". It will turn yellow when enabled. You may need to click on Advanced to get to the "Allow Plug-ins" setting.

1b. Enter your normal credentials for the Orders system and gateway as well as for the Market Data system and gateway. It is possible to mix-and-match which Rithmic system one logs into for orders and for market data.

1c. Using your favorite TCP utility, you can verify that the plug-in host is running by seeing that 127.0.0.1:3010 and 127.0.0.1:3012 are being listened to for TCP.

1. Change the configuration of your RAPI app to use the plug-in.

2a. Define two environment variables before instantiating REngine. `RAPI_MD_ENCODING=4` and `RAPI_IH_ENCODING=4`. In .NET you might use `System.SetEnvironmentVariable()`. In C++ the environment variables have to be in `REngineParams::envp`. You should be able to see these environment variables in your RAPI log file. You can also use the REngine routines available to manage environment variables, such as `REngine::setEnvironmentVariable()`, if you have already instantiated the REngine instance.

- 2b. Use "127.0.0.1:3010" as your MdCnnctPt and "127.0.0.1:3012" as your IhCnnctPt.
- 2c. When logging into your RAPI app, the credentials used for the MdCnnctPt and IhCnnctPt must match the credentials entered into R | Trader Pro for its Market Data connection.

Connecting to Multiple Rithmic Systems Simultaneously Using Environments

R | API looks for configuration information from environment variables and/or during the REngine constructor. A properly configured set of environment variables associates itself with a particular Rithmic system. However, it is possible for one instance of REngine to be associated with multiple Rithmic systems. This enables users to connect to one system for their market data, and another system for their trading.

Note : The discussion which follows assumes a familiarity with the idea of environment variables (name value pairs) that exist in operating system process spaces.

Background

Historically, one set of environment variables was conveyed to the REngine constructor via REngineParams, either as an array of C-style strings (in C++) or as properties of REngineParams (in .NET). The REngine would also examine the process space for additional environment variables. With this approach, each instance of REngine was associated with a single environment, and consequently, a single Rithmic system.

Multiple Environments

However, there are cases where one instance of REngine may need to connect to multiple Rithmic systems. For example, a user may have a market data subscription in Rithmic System A, with a trading account on Rithmic System B. Having the ability to connect simultaneously to system A for market data and system B for placing orders would remove the need for another (redundant and/or costly) market data subscription in Rithmic System B.

To support these use cases, the REngine class now supports multiple environments. Each environment is keyed with an identifying string. There is a default key of "system" for the default environment, taken during the REngine constructor. When logging in, one can specify which environment to use with a connect point. The design intent is for an environment to correspond to a particular gateway of a Rithmic system.

When specifying keyed environments when logging in, the environment will be validated for the presence of the minimal number of connection-related variables.

Environment Variable	C++	.NET
MML_DMN_SRVR_ADDR	required	required, corresponds to REngineParams.DmnSrvrAddr
MML_DOMAIN_NAME	required	required, corresponds to REngineParams.DomainName
MML_LIC_SRVR_ADDR	required	required, corresponds to REngineParams.LicSrvrAddr
MML_LOC_BROK_ADDR	required	required, corresponds to REngineParams.LocBrokAddr
MML_LOGGER_ADDR	required	required, corresponds to REngineParams.LoggerAddr
MML_LOG_TYPE	required	not used
MML_SSL_CLNT_AUTH_FILE	required	not used
USER	required	not used

Prior Behavior With A Single Environment

If there is no need to use multiple environments, one can specify an empty string for the environment key when logging in. This approach will mirror behavior prior to the introduction of multiple environment functionality.

Comments

- R | API programmers, especially vendors, may wish to keep any environment-related configurations outside of their application binary, as these configurations can change over time. Configuration files are an example of this approach.
- When connecting to a plug-in host, the environment does not matter, as the connect point is on the loopback adapter. However, the user/password credentials must match the credentials entered into the plug-in host.
- Setting an operating system process space's environment variable while an R | API app is running will not cause that environment variable to appear in the REngine's default system environment. When in doubt, verify that the desired set of environment variables and values exist using the REngine routines.

Programming Interface

Programmers can use the following routines to list/get/set/unset environment variables. These routines require an REngine instance, so a default environment passed via REngineParams will always exist. Each environment is keyed by a string. The default environment from REngineParams has a key of "system".

```
RApi::REngine::listEnvironments()  
RApi::REngine::getEnvironment()  
RApi::REngine::setEnvironmentVariable()  
RApi::REngine::unsetEnvironmentVariable()  
RApi::sDEFAULT_ENVIRONMENT_KEY (c++) or Constants.DEFAULT_ENVIRONMENT_KEY (.NET)
```

Connections

Applications built on Rithmic APIs establish multiple connections to the Rithmic trading platform. Some connections are transient, while others are persistent. The transient connections can appear as environment variables or values in REngineParams. Persistent connections are specified as connect points.

Connect Points

Above, we learned that there are multiple installations of the R | Trade Execution Platform, i.e. the different Rithmic systems. We also **learned** that each environment is associated with a particular Rithmic system.

A connect point is a logical name for a network address within a Rithmic system. Each connect point exposes a different set of services. More explicitly, the connect point + environment yields an IP and port. The environment drives how the connect point is resolved, so the same connect point string value might resolve differently if paired with a different environment.

Connect points are also where user credentials are submitted. Because a connect point is associated with a particular Rithmic system, the user id and password passed into that connect point must also match the associated Rithmic system. In other words, logging in to a specific system requires an environment, a connect point, a user and a password.

An exception to how credentials are handled is when connecting to a plug-in host. When connecting to a plug-in host, the user id and password must match what was entered for the plug-in host, regardless of the environment specified. Plug-in host connect points differ in that they are in the IP:port form and do not need to be resolved.

Another exception is the administrative connect point specified in REngineParams. The administrative connect point does not use credentials.

The following connect points are used in R | API :

- **RApi::LoginParams::sMdCnnctPt** - Connect point for the ticker plant. This connect point is required for real-time market data services.
- **RApi::LoginParams::sTsCnnctPt** - Connect point for the order plant, providing order handling services.
- **RApi::LoginParams::sPnlCnnctPt** - Connect point for the pnl, providing pnl snapshots and updates. The environment used for this connect point is always the same as the environment used for the order plant.
- **RApi::LoginParams::sIhCnnctPt** - Connect point to history plant for historical (not real-time) market data such as candlestick/bar data.

If one does not specify a particular connect point, a connection to that sub-system will not be established, and access to the services provided by that segment of R | Trade Execution Platform will be unavailable to the REngine. The most common error returned in this case will be a 'no handle' error.

Connect points available for a particular Rithmic system are distributed by the Rithmic operations team in a 'connection_params.txt' file. These text files are generally distributed via the user's download directory on the Rithmic webserver. See the above discussion on **Configuring A Rithmic API Application** for more information.

Aggregated Data Market Data

Rithmic installations may also offer aggregated market data feeds. End users with limited end-to-end network bandwidth or other computing limitations may prefer using these aggregated market data feeds for an improved user experience.

With aggregated market data, updates to certain types of data are accumulated and periodically the most recent version of that data is published. The publishing frequency may depend on the Rithmic system's configuration, but is generally set to 0.25 seconds. So, for example, if the best bid/ask prices or quantities change many times over the 0.25 second interval, only the most recent values are published. There are also types of data which are not aggregated, such as trades. Trades pass through without any aggregation.

To make use of an aggregated market data feed, one would use the market data connect point for the aggregated feed instead of for the normal/raw market data feed. In other words, when logging in, one changes the market data connect point value to switch between aggregated and non-aggregated market data feeds.

Aggregated market data connect point values are also found in the connection_params files.

Gateways and Points-Of-Presence

Some Rithmic installations have multiple "Gateways". A gateway is a geographically local point-of-presence providing access to a particular Rithmic system. There may be multiple gateways to a single Rithmic system. These gateways may be spread throughout the world, such as in North America, South America, Europe and Asia. Some end users find that using a different (perhaps geographically closer?) gateway improves their user experience.

For example, in Europe, an end user might find that the Frankfurt gateway offers the best performance/user experience. The very best performance can be obtained by colocating your program with Rithmic. Colocation

services are available from our sister company, [TheOmne.net](#). When you colocate, you use a LAN-only gateway, such as Rithmic 01 Colo 75 or Rithmic Paper Trading Colo 75.

Best Practices

Because there are many Rithmic systems, each with multiple gateways, application developers often find that keeping connection point settings (i.e. the parameters to the REngine constructor and to REngine::login()) configurable, and outside of their application binary.

In this way, one can add support for new Rithmic systems and/or gateways with a configuration change, rather than rebuilding a binary.

Connection Management

Updates to the status of these connections are provided via RCallbacks::Alert() in R | API. The alert types associated with connections are :

- Connection Opened - a connection has been established.
- Connection Broken - a connection has been broken or is in recovery. When a connection is broken, R | API will attempt to automatically recover the connection. Subscriptions will be resubmitted automatically as part of the recovery processing.
- Connection Closed - a connection has been closed.
- Quiet Heartbeat - a heartbeat was not detected. Heartbeats are used to verify the integrity of each connection. It is possible for the quiet heartbeat alert to fire on a good connection if the machine could not run the heartbeat detection routine in time (i.e. cpu too busy, stopped in debugger, etc.).
- Login Complete - login credentials were received and accepted.
- Login Failed - login credentials were received but not accepted.
- Forced Logout - the session is about to be terminated because the maximum number of concurrent sessions of this user have been exceeded. If one connection receives a forced logout alert, the RAPI will log out from all connections in that login session. Users can ask their administrator to increase the maximum number of concurrent sessions.

Quiet Heartbeat vs. Connection Broken Alerts

Sometimes a connection will go bad, but the operating system will continue waiting for some timeout period (perhaps optimistically, and perhaps in consideration of very low bandwidth or high latency network connections). In these cases where the connection is truly bad, but with the OS still waiting for the timeout, you may get one or more quiet heartbeat alerts. If (when?) the OS does end up concluding the connection is bad, it will inform the process, which will appear to you via the R | API as a connection broken event. Upon receiving a connection broken event, the R | API will automatically attempt to recover the connection. It will not attempt recovery upon receipt of a quiet heartbeat event. (FYI : It will also not attempt recovery when a login fails authentication either.)

Connecting During A System Reboot

The infrastructure has a schedule to reboot at regular maintenance intervals. Usually this will happen each weekend. When the infrastructure is brought down, R | API client connections to the infrastructure will be broken, triggering connection broken alerts. Although it is possible that the broken connections will recovery automatically and gracefully, it is also possible that a recovery attempt will result in a login failure, despite valid credentials being presented. This failure will happen when the connection points accept connections before the user id/authentication service is on-line. When authentication cannot be verified, login will fail by default. So, it may be necessary to re-initiate logging in after system reboots.

Establishing And Maintaining State With The Snapshot-Update Pattern

Establishing And Maintaining State

When first connecting, programs generally need to know the current state of affairs in some area of interest, such as the current order book, or recent orders placed on a particular account. Sometimes this process of figuring out the current state of affairs is called 'establishing state'. Once a program establishes state, it may want to be notified of any updates, or changes to the state of affairs. Keeping track of updates is sometimes referred to as 'maintaining state'. The RAPI provides methods designed to establish and maintain state, by providing methods which retrieve snapshots of data as well as methods which subscribe for updates to that data.

Polling Snapshots

Although it is tempting to continually poll the system by requesting snapshots over and over to keep track of the current state of affairs, doing so is much slower and resource inefficient than using the snapshot-update pattern. The polling approach causes chunks of the database to be transmitted to the RAPI client over and over, even if the data has not changed. There are also limits to how often an application can poll the infrastructure, and exceeding the limits can result in the suspension of services.

Poll Once, Then Updates

The recommended approach to establishing and maintaining state is to apply the snapshot-update pattern. The basic idea of this pattern is to retrieve a snapshot of the state of affairs, and subsequently apply updates to that snapshot. By applying these updates to the snapshot, the application always has the current information. Because only information which has changed is transmitted, the increased efficiency can result in dramatic changes in application responsiveness over the polling approach.

Snapshots are provided by databases within the infrastructure, and updates are published as part of the real-time data stream. Most of the RAPI methods which retrieve snapshots begin with 'replay', and most of the methods which subscribe for updates begin with 'subscribe'.

This pattern of snapshots and updates is designed to be efficient and flexible, at the cost of minor complexity.

An Example With Orders

An example of the snapshot-updates pattern is when after successfully logging in to the trading system connect point, an application wants to display all orders under an account from today's trading session. In addition, while connected, the application wants to receive any updates to these orders as they occur. The application should first call `REngine::subscribeOrder()`. This call causes any order updates for the account to be sent to the app. Next, the application should call `REngine::replayAllOrders()`. This second call requests snapshots of all orders that were active this trading session. As updates are received, they can be applied to the snapshot, with the result providing the current state of affairs. If an update is received before the snapshot arrives, it should be cached, and then applied to the snapshot when it does arrive.

Some Subtleties To Be Aware Of

There are subtleties to this pattern of snapshots and updates. It is important to call the request for updates first, as it is possible an update is published after the snapshot has been requested, but before updates have been requested. If this happens, the update published in between the two calls will be missed if the snapshot was requested first.

Another subtlety is that it is that the snapshot may incorporate an update. In this case, the update was processed by the database before the snapshot request was processed but after the subscription request was processed. By applying any updates that arrived prior to the snapshot in the same sequence that the updates arrived, the resultant state of affairs should be correct and current.

Finally, when subscribing to market depth by price (REngine::subscribe() with the market depth flag enabled), the intricacies of the snapshot-update pattern are internal to the RAPI, so there is no need to explicitly request an order book snapshot.

See also

REngine::replayAllOrders() and REngine::subscribeOrder()

REngine::replayBrackets() and REngine::subscribeBracket()

REngine::replayExecutions() and REngine::subscribeOrder()

REngine::replayOpenOrders() and REngine::subscribeOrder()

REngine::replayPnl() and REngine::subscribePnl()

Strings And tsNCharcb (C++ Only)

This section provides an overview of how the tsNCharcb data type is used.

Why use the tsNCharcb for representing strings?

The tsNCharcb is the standard string data structure native to the larger body of work used by Rithmic. It is a lightweight structure compatible with both C and C++ and is used by the libraries the R | API is written on.

Because it includes both a pointer and a length, many common operations can be done more efficiently than with char * strings. For example, when comparing strings, one can test the length of each string first, potentially saving a memory comparison call.

The C++ string type shares some of the characteristics and efficiencies of the tsNCharcb, but the tsNCharcb data structure and its associated body of work pre-date the C++ strings. As a result, the R | API also uses the tsNCharcb instead of the C++ string.

A tsNCharcb represents the absence of a value when the pData is equal to NULL and iDataLen is equal to 0 (zero). Sometimes we will refer to such a tsNCharcb as 'empty'.

When a routine has a pointer to a tsNCharcb as a parameter, and you do not wish to specify that parameter, you can set the pointer value to null, or the tsNCharcb members to null and zero.

When a routine asks for an array of tsNCharcb, you should provide the array, with each array element being null or containing an empty tsNCharcb.

How does one convert a C++ string into a tsNCharcb?

```
string strCpp;
tsNCharcb sNChar;

...

if (strCpp.empty())
{
    sNChar.pData = NULL;
    sNChar.iDataLen = 0;
}
else
{
    sNChar.pData = (char *)strCpp.data();
```

```
sNChar.iDataLen = strCpp.length();  
}
```

How does one convert a tsNCharcb into a C++ string?

```
string strCpp;  
tsNCharcb sNChar;  
  
...  
  
strCpp = string(sNChar.pData, sNChar.iDataLen);
```

Do I need to initialize both the pData and the iDataLen of each tsNCharcb?

Yes. The two should always be consistent. If the string is empty/undefined, the pData should be set to NULL, and the iDataLen should be set to 0 (zero).

If the two values are inconsistent, the behavior of the api is undefined. In the best case, you will get a response of API_BAD_INPUT for some call. In the worst case, your application's memory space will be 'corrupt'.

It is possible to have pData pointing to a legitimate buffer, with iDataLen set to zero.

Who owns the memory pointed to by tsNCharcb::pData?

Whoever allocated the memory is responsible for it. When passing tsNCharcb parameters to the R | API methods, the data will be copied within the method before returning. When the R | API invokes user-defined callbacks, the **tsNCharcb::pData** memory will most likely become invalid immediately upon exiting the callback method, so if the values will be kept, they need to be copied.

On Timestamps (ssboe, usecs and nsecs)

The Rithmic APIs make use of unix time conventions for timestamps. Unix time is the number of Seconds Since the Beginning Of the Epoch (ssboe). The epoch is defined to have begun on Jan 1, 1970 UTC (Universal Coordinated Time). One can find more detailed discussions on unix time on the Internet, as well as documentation on how one might convert unix time to/from other time formats using a variety of technologies.

There is often a microseconds (usecs) portion of the timestamp to accompany the ssboe value. Sometimes there is even a nanoseconds (nsecs) portion.

Both the ssboe, usecs and nsecs values are usually stored as integer types.

On Threads

Threads In C++ API

The C++ version of the Rithmic APIs will create an internal "worker" thread. Almost all of the callbacks will be invoked on this worker thread. The reason for this arrangement is because the methods exposed by REngine are typically handled asynchronously by distributed services. When the distributed service responds to REngine's methods, the worker thread will invoke the appropriate callback. In this way the main thread is not blocked.

If you use multiple threads and mutexes in your application space, it is possible to deadlock, as R | API also uses mutexes internally. Generally, the Rithmic APIs will lock upon entry into an REngine method, as well as prior to invoking a user-defined callback. You can use the very mutex R | API uses by calling REngine::lock() and REngine::unlock() to avoid these deadlock situations preserving the order of the lock() calls in each thread.

Threads In .NET API

The .NET version of the Rithmic APIs creates multiple threads. Each persistent network connection will have an input thread as well as an output thread. There are other administrative threads which may also be created. The .NET APIs do not expose `REngine::lock()` and `REngine::unlock()` methods because thread management is done differently than in the C++ APIs.

Error Handling Conventions

Error Codes

All Rithmic APIs make use of the same set of integer error codes to convey information about error conditions. Rithmic APIs also make use of these same exception classes internally.

In C++, the set of error codes are defined in the `RApi.h` header file, starting with `API_OK`.

Exception Classes

The exception class is **`OmneException`**. Error codes can be retrieved from exceptions using **`OmneException.getErrorCode()`**.

The 'aiCode' Out Parameter (C++ only)

All C++ Rithmic API methods provide a return value (`OK` or `NOT_OK`, defined in `RApi.h`), where `NOT_OK` indicates an error condition. When an error is encountered, the `aiCode` 'out' parameter is set with the value of the appropriate error code.

Why And Where Errors Might Occur

Typically, an 'action' object such as `REngine`, requests services from the network, resulting in a callback method being invoked to convey the response. Errors can occur :

- Within the requesting method as it tries to form the request and send it to the infrastructure. In this case, the error will result in an error code returned directly from the method being called.
- Within the infrastructure as it tries to service the request. In this case, the error code will be conveyed via the standard callback associated with the originating method and possibly `RCallbacks::Alert()`, depending on the severity of the error. The 'Info' classes have an `iRpCode` field when they need to convey an error status.
- Within the API as it processes the response and invokes the appropriate callback. This case is similar to conveying errors encountered in the infrastructure. The error code will be conveyed via the standard callback associated with the originating method and possibly `RCallbacks::Alert()`, depending on the severity of the error.

Constructors and destructors (caution when using with STL) may throw exceptions also.

Native exceptions being thrown from a Rithmic API method indicate unhandled exceptions.

Examples Of Errors

- An example of an error encountered locally in the `REngine` is if one attempts to subscribe to market data without being logged in. The error code returned would be 11, or 'no handle', indicating that there was no handle for the market data connection.

- Alerts are fired if a more serious error has been reported by R | Trade Execution Platform. An example would be an error code of 14, or 'unknown request'. This error code indicates that the service being requested is down or off-line.
- When an expected error is returned by R | Trade Execution Platform, the response code of the 'Info' instance, which is passed to the callback method, will contain the corresponding error code. An example of this case would be if `REngine::getRefData()` were called for an invalid instrument. `RCallbacks::RefData()` would be called where `RefDataInfo`'s response code would contain a value of 7, or 'no data'. In other words, R | Trade Execution Platform has no data on the instrument being referenced.

When an unexpected error is returned by R | Trade Execution Platform, an Alert is raised by `RCallbacks::Alert()`. The `AlertInfo` instance will contain information about the error.

Error Handling In User Implemented Callbacks

When implementing a callback method in C++, the expectation is that the callback will return (OK), or return (NOT_OK) with the value of `*aiCode` set to `API_IGNORED`. If the callback method returns NOT_OK with a `*aiCode` value that is not `API_IGNORED`, the error will be interpreted as a grievous error and cause the connection associated with the callback to be broken. This should only be done when actually encountering a grievous error.

Basic Steps of a Program that Incorporates R | API

The following list presents the basic steps that a program that incorporates R | API must perform:

- Create an instance of `AdmCallbacks`. This implies that you have to define your own class, since `AdmCallbacks` is abstract.
- Create and populate an instance of `REngineParams`. This set of parameters contains the configuration information needed to instantiate the `REngine`. Some of the configuration information will be retrieved from the environment (`REngineParams::envp`). Actual configuration values will be provided by the Rithmic operations team and may be specified in a configuration file (such as "uat_connection_params.txt") in your download directory. The naming conventions used in the operations document will loosely match the names in R | API interface.
- Create an instance of the `REngine`, using the `AdmCallbacks` and `REngineParams` from above. The reason you need the `AdmCallbacks` is because you can receive administrative alerts without being logged in.
- Create an instance of `RCallbacks`. Again, because it is an abstract class, you must derive your own subclass and implement it. Implementing all callbacks forces the user to explicitly handle or ignore the available callbacks.
- Log in using `REngine::login()`. The methods invoked on the `RCallbacks` instance passed into the login call will correspond to this login session.
- Wait for updates on the login status by examining the `RCallbacks::Alert()` callback.
- If the login completes for all connect points, the program will be able to access the services of R | Trade Execution Platform. If any of the logins fail for any of the connect points, R | API will automatically log out of all connect points. In other words, each connect point has its own authentication handshake, and if one fails, they all fail.
- Assuming a successful login, invoke methods on `REngine`, process responses in `RCallbacks` as necessary.
- Logout. Note that `REngine::logout()` will automatically be called in the `REngine` destructor so this step is optional if you intend on destroying the `REngine` instance immediately. However, if you call `REngine::logout()` yourself, you can log back into R | Trade Execution Platform using the same instance of the `REngine`, but perhaps with different credentials.

- Destroy the RCallbacks instance. You should not destroy the callbacks instance before logging out, as any of its methods may be called until the logout is complete.
 - Destroy the REngine instance.
 - Destroy the AdmCallbacks instance.
-

Feedback And Bug Reports

We are always interested in hearing feedback regarding the API. This ranges from feature requests to bug reports to typos in the documentation.

Contact Info

We can be reached at rapi@rithmic.com.